

Kmeans++

The Dataset (6 Points, 3 Features)

Point	x	y	z
P_1	1	1	2
P_2	1	2	1
P_3	8	8	9
P_4	9	8	8
P_5	1	9	2
P_6	2	8	1



Export to Sheets



Phase 1: K-Means++ Initialization

The goal is to pick $k = 3$ initial centers.

Step 1: Pick μ_1 We pick one point at random from the dataset. Let's say we pick P_1 .

- $\mu_1 = (1, 1, 2)$

Step 2: Pick μ_2 We calculate the squared Euclidean distance $D(x)^2$ from every point to the nearest center (which is currently just μ_1). $D(x, \mu)^2 = (x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + (x_3 - \mu_3)^2$

Point	Calculation vs $\mu_1(1, 1, 2)$	$D(x)^2$	Probability $P(x) = \frac{D(x)^2}{\sum D(x)^2}$
P_1	$(1 - 1)^2 + (1 - 1)^2 + (2 - 2)^2$	0	$0/413 = 0$
P_2	$(1 - 1)^2 + (2 - 1)^2 + (1 - 2)^2 = 0 + 1 + 1$	2	$2/413 \approx 0.005$
P_3	$(8 - 1)^2 + (8 - 1)^2 + (9 - 2)^2 = 49 + 49 + 49$	147	$147/413 \approx 0.356$
P_4	$(9 - 1)^2 + (8 - 1)^2 + (8 - 2)^2 = 64 + 49 + 36$	149	$149/413 \approx 0.361$
P_5	$(1 - 1)^2 + (9 - 1)^2 + (2 - 2)^2 = 0 + 64 + 0$	64	$64/413 \approx 0.155$
P_6	$(2 - 1)^2 + (8 - 1)^2 + (1 - 2)^2 = 1 + 49 + 1$	51	$51/413 \approx 0.123$
Sum		413	1.00



Export to Sheets



We pick the next center based on these probabilities. Since P_4 has the highest weight, let's select it.

- $\mu_2 = (9, 8, 8)$

Step 3: Pick μ_3 Now we find the distance from each point to its **nearest** center (μ_1 or μ_2).

Point	$D(x, \mu_1)^2$	$D(x, \mu_2)^2$	Min $D(x)^2$	Probability $P(x)$
P_1	0	149	0	0
P_2	2	$(1 - 9)^2 + (2 - 8)^2 + (1 - 8)^2 = 64 + 36 + 49 = 149$	2	$2/119 \approx 0.017$
P_3	147	$(8 - 9)^2 + (8 - 8)^2 + (9 - 8)^2 = 1 + 0 + 1 = 2$	2	$2/119 \approx 0.017$
P_4	149	0	0	0
P_5	64	$(1 - 9)^2 + (9 - 8)^2 + (2 - 8)^2 = 64 + 1 + 36 = 101$	64	$64/119 \approx 0.538$
P_6	51	$(2 - 9)^2 + (8 - 8)^2 + (1 - 8)^2 = 49 + 0 + 49 = 98$	51	$51/119 \approx 0.428$
Sum			119	1.00



Export to Sheets



P_5 is the most likely candidate. Let's select it.

- $\mu_3 = (1, 9, 2)$

Phase 2: Iterative K-Means (Assignment & Update)

Now we have our 3 initial centers: $\mu_1(1, 1, 2), \mu_2(9, 8, 8), \mu_3(1, 9, 2)$.

Step 4: Assignment (Iteration 1)

We calculate the distance from every point to all three centers and assign it to the minimum.

- $P_1(1, 1, 2)$: Dist to $\mu_1 = 0$. **Assign to Cluster 1.**
- $P_2(1, 2, 1)$:
 - to μ_1 : $\sqrt{0 + 1 + 1} = 1.41$
 - to μ_2 : $\sqrt{64 + 36 + 49} = 12.2$
 - to μ_3 : $\sqrt{0 + 49 + 1} = 7.07$
 - Assign to Cluster 1.**
- $P_3(8, 8, 9)$:
 - to μ_1 : $\sqrt{49 + 49 + 49} = 12.1$
 - to μ_2 : $\sqrt{1 + 0 + 1} = 1.41$
 - to μ_3 : $\sqrt{49 + 1 + 49} = 9.95$
 - Assign to Cluster 2.**
- $P_4(9, 8, 8)$: Dist to $\mu_2 = 0$. **Assign to Cluster 2.**
- $P_5(1, 9, 2)$: Dist to $\mu_3 = 0$. **Assign to Cluster 3.**
- $P_6(2, 8, 1)$:

- to μ_1 : $\sqrt{1 + 49 + 1} = 7.14$

- **Assign to Cluster 3.**

Step 5: Update (Recalculating Centers)

We find the mean of the points assigned to each cluster.

- **New μ_3** (from P_5, P_6): $x = \frac{1+2}{2} = 1.5$; $y = \frac{9+8}{2} = 8.5$; $z = \frac{2+1}{2} = 1.5 \rightarrow$
(1.5, 8.5, 1.5)

Summary of Results

Cluster	Initial Center (K-means++)	Final Center (After 1 Iteration)	Points Assigned
1	(1, 1, 2)	(1, 1.5, 1.5)	P_1, P_2
2	(9, 8, 8)	(8.5, 8, 8.5)	P_3, P_4
3	(1, 9, 2)	(1.5, 8.5, 1.5)	P_5, P_6

Export to Sheets



PK

YouTube

Q

🎤

⋮

👤

Sign in

KMeans++ Cluster Initialization ($k = 4$)

1. Choose one center c_1 randomly from X .

2. Choose a new center c_i from X using probability distribution weights $P(x_n)$.

$$P(x_n) = \frac{D(x_n)}{\sum_{n=1}^N D(x_n)}$$

$$D(x_n) = \min_{j < i} \|x_n - c_j\|^2$$

3. Repeat step 2 until k centers have been selected

▶ ◀ 0:00 / 1:15

⚙️

🔍

📺

↗️

K-Means++ Centroid Initialization



Jacob Bumgarner

201 subscribers

Subscribe



167



Share



Save



8.1K views 3 years ago

No description has been added to this video.

...more

Agglomerative clustering

1. The Dataset (5 Points, 3 Features)

Point	x	y	z
A	1	1	1
B	1	2	1
C	7	8	7
D	8	8	8
E	1	8	1



Export to Sheets



Step 1: Initial Distance Matrix (M_0)

We use the **Squared Euclidean Distance**: $dist^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2$.

Calculations:

- AB**: $(1 - 1)^2 + (2 - 1)^2 + (1 - 1)^2 = 0 + 1 + 0 = \mathbf{1}$
- AC**: $(7 - 1)^2 + (8 - 1)^2 + (7 - 1)^2 = 36 + 49 + 36 = \mathbf{121}$
- AD**: $(8 - 1)^2 + (8 - 1)^2 + (8 - 1)^2 = 49 + 49 + 49 = \mathbf{147}$
- AE**: $(1 - 1)^2 + (8 - 1)^2 + (1 - 1)^2 = 0 + 49 + 0 = \mathbf{49}$
- BC**: $(7 - 1)^2 + (8 - 2)^2 + (7 - 1)^2 = 36 + 36 + 36 = \mathbf{108}$
- BD**: $(8 - 1)^2 + (8 - 2)^2 + (8 - 1)^2 = 49 + 36 + 49 = \mathbf{134}$
- BE**: $(1 - 1)^2 + (8 - 2)^2 + (1 - 1)^2 = 0 + 36 + 0 = \mathbf{36}$
- CD**: $(8 - 7)^2 + (8 - 8)^2 + (8 - 7)^2 = 1 + 0 + 1 = \mathbf{2}$
- CE**: $(1 - 7)^2 + (8 - 8)^2 + (1 - 7)^2 = 36 + 0 + 36 = \mathbf{72}$
- DE**: $(1 - 8)^2 + (8 - 8)^2 + (1 - 8)^2 = 49 + 0 + 49 = \mathbf{98}$

Matrix M_0 (Upper Diagonal):

	A	B	C	D	E
A	-	1	121	147	49
B		-	108	134	36
C			-	2	72
D				-	98
E					-



Export to Sheets



Step 2: First Merge (A and B)

The minimum distance is **1**. We merge **A** and **B** into cluster **(AB)**. Using **Single Linkage**, the

distance to (AB) is the **minimum** distance to either A or B.

Calculations for new row/column **(AB)**:

- $\text{dist}((AB), C): \min(AC, BC) = \min(121, 108) = \mathbf{108}$
- $\text{dist}((AB), D): \min(AD, BD) = \min(147, 134) = \mathbf{134}$
- $\text{dist}((AB), E): \min(AE, BE) = \min(49, 36) = \mathbf{36}$

Matrix M_1 (Upper Diagonal):

	(AB)	C	D	E
(AB)	-	108	134	36
C		-	2	72
D			-	98
E				-

 Export to Sheets 

Step 3: Second Merge (C and D)

The minimum distance is **2**. We merge **C** and **D** into cluster **(CD)**.

Calculations for new row/column **(CD)**:

- $\text{dist}((CD), (AB)): \min(C(AB), D(AB)) = \min(108, 134) = \mathbf{108}$
 $\min(AC, AD, BC, BD) = \min(121, 147, 108, 134) = 108$
- $\text{dist}((CD), E): \min(CE, DE) = \min(72, 98) = \mathbf{72}$

Matrix M_2 (Upper Diagonal):

	(AB)	(CD)	E
(AB)	-	108	36
(CD)		-	72
E			-

 Export to Sheets 

Step 4: Third Merge (AB and E)

The minimum distance is **36**. We merge cluster **(AB)** and point **E** into cluster **(ABE)**.

Calculations for new row/column **(ABE)**:

- $\text{dist}((ABE), (CD)): \min((AB)(CD), E(CD)) = \min(108, 72) = \mathbf{72}$
 $\min(AC, AD, BC, BD, EC, ED) = (121, 147, 108, 134, 72, 98) = 72$

Matrix M_3 (Upper Diagonal):

	(ABE)	(CD)
(ABE)	-	72
(CD)		-

 Export to Sheets 

Final Result: The last two clusters **(ABE)** and **(CD)** merge at a distance of **72**.

Elbow Method

Cluster	Points Assigned	Final Center (μ)
Cluster 1	$P_1(1, 1, 2), P_2(1, 2, 1)$	$\mu_1 = (1, 1.5, 1.5)$
Cluster 2	$P_3(8, 8, 9), P_4(9, 8, 8)$	$\mu_2 = (8.5, 8, 8.5)$
Cluster 3	$P_5(1, 9, 2), P_6(2, 8, 1)$	$\mu_3 = (1.5, 8.5, 1.5)$

 Export to Sheets 

2. Step-by-Step Distance Calculations

We calculate the squared distance $||x - \mu||^2$ for every point to its own cluster center.

Cluster 1 Calculations

- P_1 to μ_1 : $(1 - 1)^2 + (1 - 1.5)^2 + (2 - 1.5)^2 = 0 + 0.25 + 0.25 = \mathbf{0.5}$
- P_2 to μ_1 : $(1 - 1)^2 + (2 - 1.5)^2 + (1 - 1.5)^2 = 0 + 0.25 + 0.25 = \mathbf{0.5}$
- Cluster 1 Sum:** $0.5 + 0.5 = \mathbf{1.0}$

Cluster 2 Calculations

- P_3 to μ_2 : $(8 - 8.5)^2 + (8 - 8)^2 + (9 - 8.5)^2 = 0.25 + 0 + 0.25 = \mathbf{0.5}$
- P_4 to μ_2 : $(9 - 8.5)^2 + (8 - 8)^2 + (8 - 8.5)^2 = 0.25 + 0 + 0.25 = \mathbf{0.5}$
- Cluster 2 Sum:** $0.5 + 0.5 = \mathbf{1.0}$

Cluster 3 Calculations

- P_5 to μ_3 : $(1 - 1.5)^2 + (9 - 8.5)^2 + (2 - 1.5)^2 = 0.25 + 0.25 + 0.25 = \mathbf{0.75}$
- P_6 to μ_3 : $(2 - 1.5)^2 + (8 - 8.5)^2 + (1 - 1.5)^2 = 0.25 + 0.25 + 0.25 = \mathbf{0.75}$
- Cluster 3 Sum:** $0.75 + 0.75 = \mathbf{1.5}$

3. Total Within-Cluster Sum of Squares

The total WCSS is the sum of the variations within each individual cluster.

$$WCSS = \text{Sum}(C_1) + \text{Sum}(C_2) + \text{Sum}(C_3)$$

$$WCSS = 1.0 + 1.0 + 1.5$$

WCSS = 3.5

Summary Table: Which to use when?

Feature	K-Means	K-Means++	Agglomerative
Speed	Very Fast	Fast	Slow on large data
Cluster Shapes	Spherical only	Spherical only	Flexible (Linkage dependent)
Outlier Sensitivity	High	High	Moderate to High
Initialization	Random (Risky)	Smart (Stable)	None needed
Best For...	Huge datasets	Production-level K-Means	Small data, discovering hierarchy

		gold labels			
		urgent	normal	spam	
system output	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Figure 6.5 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2) , how many documents from c_1 were (in)correctly assigned to c_2

Micro-precision

		Actual	
Prediction	T	T	F
	F		
T	268	99	
F	99		$340 + 83 + 212 = 635$

1. Sensitivity (True Positive Rate / Recall)

Sensitivity measures the model's ability to correctly identify actual positive cases.

- **The Question:** "Of all the people who *actually* have the condition, how many did the model correctly catch?"
- **The Focus:** Minimizing **False Negatives** (FN).
- **High Sensitivity means:** You rarely miss a positive case. This is critical in medical screenings where missing a disease is dangerous.

2. Specificity (True Negative Rate)

Specificity measures the model's ability to correctly identify actual negative cases.

- **The Question:** "Of all the people who *actually* do not have the condition, how many did the model correctly identify as negative?"
- **The Focus:** Minimizing **False Positives** (FP).
- **High Specificity means:** You rarely give "false alarms." This is critical when a positive result leads to expensive or invasive follow-up treatments.

3. The Formulas

Both metrics are calculated using the values from a standard 2×2 confusion matrix.

- **Sensitivity:**

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

- **Specificity:**

$$\text{Specificity} = \frac{TN}{TN + FP}$$

False positive Rate = $1 - \text{specificity}$

True positive Rate = $\text{sensitivity} / \text{Recall}$

ROC Curve (Receiver Operating Characteristic)



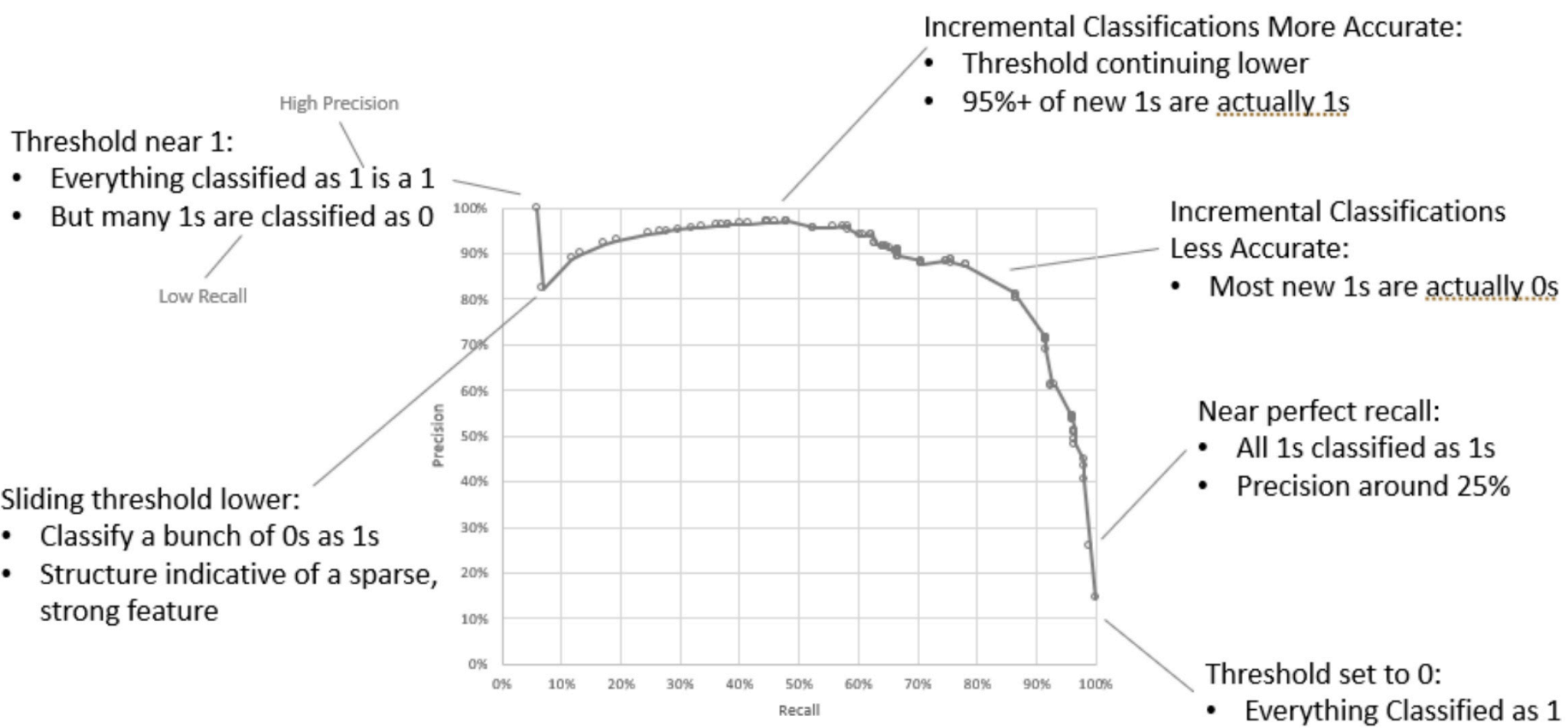
1. The Relationship: Success vs. Error

The y-axes in your images represent two sides of the same coin:

- **True Positive Rate (TPR / Sensitivity):** This is the "hit rate." It measures what percentage of positive cases the model correctly caught.
- **False Negative Rate (FNR / Miss Rate):** This is the "error rate." It measures what percentage of positive cases the model missed.

Mathematically, the relationship is $TPR = 100\% - FNR$.

Precision Recall Curves – PR Curves



1. The "Exclusivity" Phase (Top Left)

- **The Scenario:** You have set a very high bar for what counts as a "1."
- **The "Why":** The model is effectively saying, "I will only call this a 1 if I am 99.9% certain."
- **The Result:** You achieve **High Precision** because you aren't guessing, but you suffer from **Low Recall** because you are ignoring most of the "1s" that look even slightly ambiguous.

2. The "Efficiency" Phase (The Peak/Flat section)

- **The Scenario:** You start lowering the threshold, allowing more items through the filter.
- **The "Why":** You are picking the "low-hanging fruit." These are the clear-cut "1s" that the model missed earlier but can now identify easily.
- **The Result:** Your **Recall** jumps significantly (moving right), but your **Precision** barely drops. This is the most efficient part of your model because you are gaining many new correct answers for a very small "price" in errors.

3. The "Desperation" Phase (The Sharp Drop)

- **The Scenario:** You continue lowering the threshold even further.
- **The "Why":** You have already found all the obvious "1s." To find the remaining few, the model has to start accepting items that look very similar to "0s."
- **The Result:** Every time you gain a little more **Recall**, you are flooded with a massive amount of "0s" that the model is now misclassifying as "1s". The **Precision** plummets because your "net" is now catching more trash than fish.

4. The "Total Recall" Phase (Bottom Right)

- **The Scenario:** You have set the threshold to zero.
 - **The "Why":** You have told the model to classify **everything** as a 1.
 - **The Result:** You have **100% Recall** (you definitely caught every single 1 in the building), but your **Precision** is essentially just the "base rate" of 1s in your dataset (in this case, about 15-25%) because your model isn't actually filtering anything anymore.
-

Step 1: The Raw Data

Imagine you have 5 test samples. You have their **Actual Labels** (1 for Positive, 0 for Negative) and the **Probability Score** your model gave them.

Sample	Actual Label	Model's Probability Score
A	1 (Positive)	0.95
B	0 (Negative)	0.80
C	1 (Positive)	0.70
D	0 (Negative)	0.40
E	1 (Positive)	0.10

 Export to Sheets



Step 2: Create All Possible Pairs

To find the AUC, you must compare **every single Positive (1)** against **every single Negative (0)**.

- Total Positives (P) = 3 (Samples A, C, E)
- Total Negatives (N) = 2 (Samples B, D)
- **Total Pairs to Check** = $P \times N = 3 \times 2 = 6$ pairs.

Step 3: Score the Pairs

For each pair, we ask: "Did the model give the Positive a higher score than the Negative?".

Pair (Pos vs Neg)	Comparison	Score
A vs B	0.95 vs 0.80	1 point (Correctly ranked)
A vs D	0.95 vs 0.40	1 point (Correctly ranked)
C vs B	0.70 vs 0.80	0 points (Incorrectly ranked)
C vs D	0.70 vs 0.40	1 point (Correctly ranked)
E vs B	0.10 vs 0.80	0 points (Incorrectly ranked)
E vs D	0.10 vs 0.40	0 points (Incorrectly ranked)

 Export to Sheets



Note: If there were a tie (e.g., both 0.50), you would award **0.5 points**.

Step 4: Calculate the Final AUC

The AUC is the average of these scores:

$$AUC = \frac{\text{Total Points}}{\text{Total Pairs}} = \frac{1 + 1 + 0 + 1 + 0 + 0}{6} = \frac{3}{6} = \mathbf{0.50}$$

1. False Positive Rate (FPR)

The False Positive Rate measures what fraction of the **actual negative cases** were incorrectly predicted as positive.

- **Formula:**

$$FPR = \frac{FP}{FP + TN}$$

- **In your matrix:**
 - **Numerator:** Look for the intersection of **Actual N / 0** and **Predicted P / 1**.
 - **Denominator:** This is the sum of the entire **Actual Negative** group (everyone who actually does not have the condition).

2. False Negative Rate (FNR)

The False Negative Rate (also known as the Miss Rate) measures what fraction of the **actual positive cases** were incorrectly predicted as negative.

- **Formula:**

$$FNR = \frac{FN}{TP + FN}$$

- **In your matrix:**
 - **Numerator:** Look for the intersection of **Actual P / 1** and **Predicted N / 0**.
 - **Denominator:** This is the sum of the entire **Actual Positive** group (everyone who actually has the condition).